

BENEMÉRITA UNIVERSIDAD AUTÓNOMA DE PUEBLA

TABLERO DE

AJEDREZ

ALUMNO: HERNÁNDEZ PRADO OSMAR JAVIER

MATRÍCULA: 202331989

PROFESOR: MIGUEL ANGEL VARGAS LOMELÍ

MATERIA: GRAFICACIÓN

4TO SEMESTRE

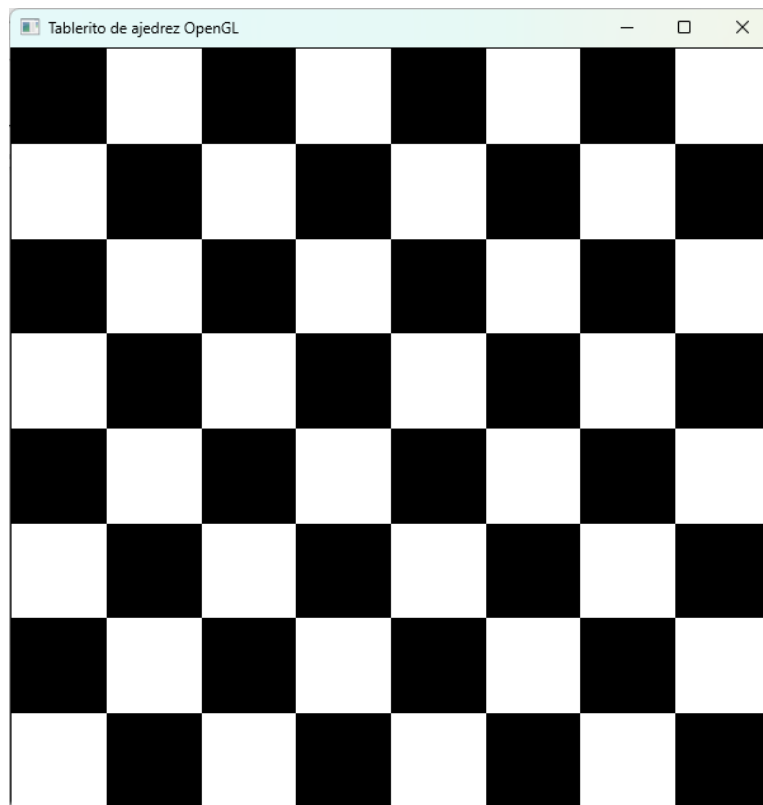
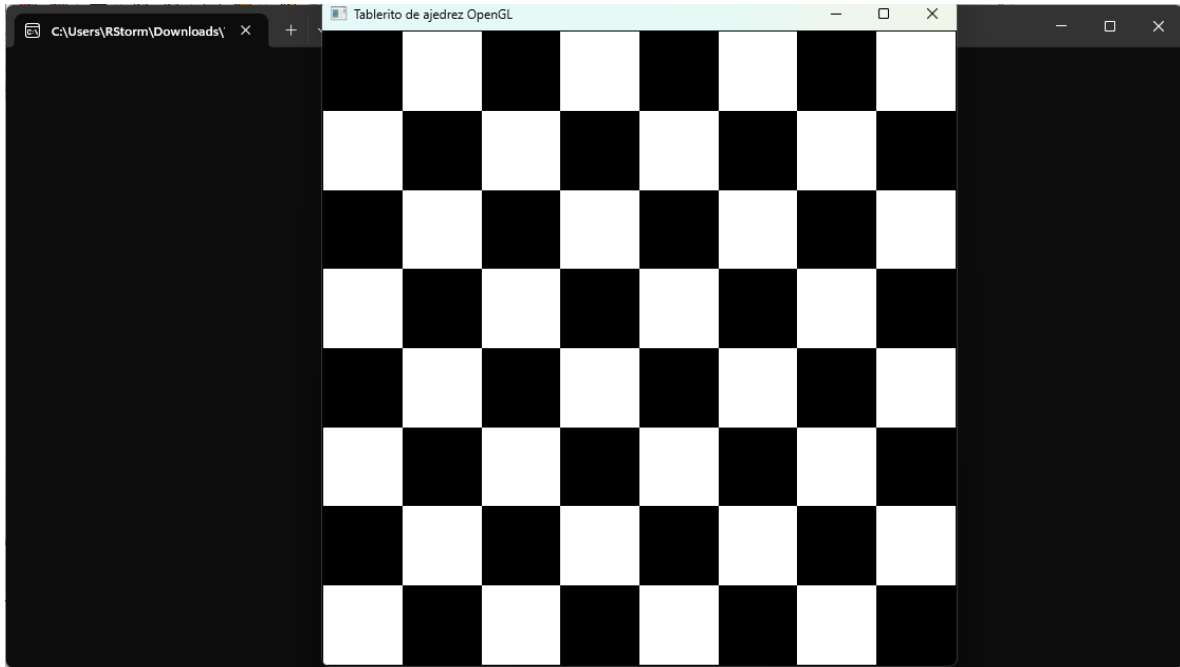
1 DE ABRIL DE 2025

PERIODO ENERO – JUNIO 2025

Descripción de la actividad:

Dibuje en OpenGL un tablero de ajedrez (de 8x8 cuadros), ocupando la mayor parte posible de su pantalla gráfica.

Capturas de pantalla:



Código:

```
#include <GL/glut.h>
```

```
void display(){
```

```
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT); //Para limpiar buffers de color y profundidad
```

```
    glLoadIdentity(); // Cargar la matriz identidad
```

```
    glTranslatef(0.0f, 0.0f, -9.7f); // Aleja la tetera para que sea visible
```

```
    float tamCasilla = 1.0f; // Tamano por casilla
```

```
    int filas = 8, columnas = 8; // Filas y columnas del tablero
```

```
    float mediaTabCol = columnas/2.0f, mediaTabFil = filas/2.0f;
```

```
    for (int i = 0; i < filas; i++){
```

```
        for (int j = 0; j < columnas; j++){
```

```
            //Dibujando cada casilla
```

```
            if ((i+j)%2 == 0){
```

```
                glColor3f(1.0f, 1.0f, 1.0f); // Color blanco
```

```
            }else{
```

```
                glColor3f(0.0f, 0.0f, 0.0f); //Color negro
```

```
            }
```

```
            glPushMatrix(); // Guardamos la matriz actual
```

```
            glTranslatef(j * tamCasilla - mediaTabCol * tamCasilla, (i * tamCasilla) - mediaTabFil * tamCasilla, 0.0f); //Dibujar de abajo hacia arriba en Y, como en 3D
```

```
            //glTranslatef(j * tamCasilla - mediaTabCol * tamCasilla, -(i * tamCasilla) + mediaTabFil * tamCasilla, 0.0f); //Esto seria en 2D
```

```
            glBegin(GL_QUADS); // Comenzamos a dibujar un cuadrado
```

```
                glVertex3f(0.0f, 0.0f, 0.0f); // Esquina inferior izquierda
```

```
                glVertex3f(tamCasilla, 0.0f, 0.0f); // Esquina inferior derecha
```

```
                glVertex3f(tamCasilla, tamCasilla, 0.0f); // Esquina superior
```

```
derecha
```

```

        glVertex3f(0.0f, tamCasilla, 0.0f); // Esquina superior izquierda
    glEnd(); // Terminamos de dibujar el cuadrado
    glPopMatrix(); // Restauramos la matriz guardada
}
}
glutSwapBuffers(); // Intercambiamos los buffers para mostrar el resultado
}

```

```

void reshape(int w, int h){
    glViewport(0, 0, w, h);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective(45.0, (double)w / (double)h, 1.0, 10.0);
    glMatrixMode(GL_MODELVIEW);
}

```

```

int main(int argc, char** argv){
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB | GLUT_DEPTH);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Tablerito de ajedrez OpenGL");

    glEnable(GL_DEPTH_TEST); // Habilitar el test de profundidad

    glutDisplayFunc(display);
    glutReshapeFunc(reshape);
    glutMainLoop();

    return 0;
}

```